

Borrador del índice de la memoria

Propuesta de estructura y contenido de la memoria del TFG, previa a la redacción en LaTeX. Para cada sección se listan los puntos que contendrá (un bullet por idea o párrafo previsto), la fuente de la que saldrá y las figuras propuestas. El objetivo es acordar el mapa antes de escribir: corregir aquí un capítulo cuesta minutos; corregirlo en la memoria, días.

La tesis en pocas líneas

NutriApp es un editor inteligente de planes nutricionales para el profesional colegiado. El nutricionista describe lo que necesita en su idioma; un LLM lo traduce a restricciones formales de un vocabulario cerrado; un solver de programación con restricciones (CP-SAT) genera el plan; un validador determinista lo audita; y el profesional revisa, ajusta y firma. La IA nunca decide: propone sobre una interfaz estable de restricciones que también admite entrada manual, de modo que el sistema completo funciona sin el LLM. El trabajo recorre el proceso completo: el refinamiento de la idea con profesionales en ejercicio, el diseño de la aplicación y de su base de datos, el desarrollo del motor de generación y de la capa de IA integrada en la plataforma, y la relación entre todas las piezas. La evaluación es empírica y con criterios pre-registrados, e incluye un hallazgo negativo documentado como resultado.

Este enfoque no fue el de partida. La primera versión del proyecto abarcaba todas las funcionalidades posibles de una plataforma que conectara al nutricionista con sus clientes. Al contrastarla con nutricionistas en ejercicio quedó claro que lo que de verdad demandan es ayuda con la tarea que más tiempo les quita, la elaboración de cada dieta a medida, apoyada en los datos de sus clientes; la centralización del resto de sus procesos suele depender de la clínica en la que trabajen, y la comunicación con el cliente es un complemento, no el núcleo. Ese feedback motivó la reformulación de los requisitos y el enfoque actual.

Capítulos de un vistazo

La plantilla TFGTeXiS admite más capítulos de los cinco actuales. La propuesta reparte el material técnico en cuatro capítulos nuevos (4 a 7) en lugar de concentrarlo todo en el actual “Descripción del trabajo”, siguiendo la estructura por temas del TFG de Adrián que me compartisteis.

#	Capítulo	Estado actual	Contenido en una línea
1	Introducción	Escrito; retoques	Motivación, objetivos y plan de trabajo real
2	Estado de la cuestión	Escrito parcial	Competidores + la dieta como problema de optimización + copilotos IA en salud
3	Análisis de requisitos	Escrito y al día	Reformulación del alcance, 29 RF y 11 RNF; retoques menores
4	Diseño del sistema	Nuevo	Arquitectura, modelo de datos, seguridad multi-rol, despliegue, metodología
5	El motor de generación	Nuevo	Formalización CP-SAT, reglas estructurales, validador, infactibilidad, API
6	El copiloto de IA	Nuevo	Traductor, validación previa, generación asíncrona, arbitraje del profesional
7	Evaluación	Nuevo	Estudio de formatos, rendimiento medido, prueba con nutricionistas
8	Conclusiones y trabajo futuro	Cáscara	Balance contra objetivos, limitaciones, líneas futuras

Los capítulos 1 y 8 llevan además su traducción al inglés (requisito de la UCM), y hay que actualizar resumen y abstract al marco final del proyecto.

Capítulo 1. Introducción

Ya escrito (motivación, objetivos, plan de trabajo). Retoques previstos:

- Limpiar el texto normativo de la plantilla que quedó al inicio del fichero.
- Reforzar la motivación con el recorrido real del proyecto: nace de una conversación con un nutricionista y de una primera versión que abarcaba todas las funcionalidades posibles; al presentarla a más profesionales y escuchar lo que de verdad demandaban, el centro pasó a ser la creación asistida de dietas (lo que más tiempo les cuesta), con la ficha del cliente como apoyo y la comunicación como complemento.
- Revisar la formulación de objetivos para alinearla con el marco final: asistente de redacción para el profesional colegiado, no generador autónomo de dietas.
- Actualizar el plan de trabajo, que hoy termina en la fase de validación con usuarios y no recoge las fases reales de la segunda mitad: base de datos, conexión de backend y frontend, motor de generación, capa de IA, panel del cliente y evaluación. Reflejar el orden que recomendasteis (crear la BD, conectarla, frontend, IA, integración final).

Fuente: capítulo actual + guía de indicaciones de las reuniones de dirección.

Capítulo 2. Estado de la cuestión

El análisis de competidores está escrito, pero es anterior al pivote y no cubre nada del enfoque técnico final. Secciones:

2.1 Herramientas para nutricionistas (escrita, revisar) - Mantener el análisis de Nutrium, Practice Better, Healthie y Cronometer Pro; revisar el posicionamiento para que compare contra el copiloto, no contra el alcance antiguo. - Añadir los asistentes de IA del sector (Nutrium AI Coach, Foodzilla AI, Practice Better AI, EatThisMuch) y sus carencias comunes: generación opaca, sin control fino del profesional, sin garantías duras sobre el resultado.

2.2 La dieta como problema de optimización (nueva) - Del problema de la dieta de Stigler (1945) y la programación lineal de Dantzig al menu planning moderno (Balintfy 1964, Petot 1998, Kahraman y Seven 2005). - Programación con restricciones: qué aporta frente a LP clásica; CP-SAT de OR-Tools y MiniZinc como herramientas de referencia del área.

2.3 LLMs como capa de lenguaje (nueva) - Salida estructurada con esquema; LLM local frente a API comercial y la dimensión RGPD del dato clínico; el patrón “el LLM traduce, el solver decide”.

Figuras: ninguna imprescindible; posible tabla comparativa de asistentes IA. Fuente: capítulo actual, el documento de investigación previa (bd-investigacion), las referencias ya recopiladas en el documento de formalización del solver (8 entradas) y las notas de estado del arte de CP aplicada a dieta.

Capítulo 3. Análisis de requisitos

El capítulo más maduro: reformulación del alcance con el triaje V1 a V2, 29 RF en bloques, 11 RNF y el copiloto como innovación clave. Retoques:

- Reescribir la entrada de la sección de reformulación para que el criterio del triaje quede anclado en el feedback de los profesionales, no en un recorte por alcance: el catálogo original (61 RF) respondía a “todo lo que una plataforma así podría hacer”; las conversaciones con nutricionistas revelaron que el valor está en automatizar la elaboración de dietas y en disponer de los datos del cliente para hacerlo, así que se redujo el catálogo para concentrar el esfuerzo donde estaba la demanda. El Anexo A conserva el catálogo original como registro de ese proceso.
- Añadir la justificación de las decisiones tecnológicas (Next.js con Server Actions, Python para el backend, Radix UI y Tailwind, Supabase como PostgreSQL gestionado), citando las reuniones de dirección como fuente.
- Añadir los RNF que la implementación convirtió en requisitos de facto: aislamiento multi-tenant verificado empíricamente, “el motor de generación funciona sin depender del LLM” (decisión de las reuniones de dirección que ordenó todo el desarrollo), y los dos que propuso la auditoría externa: explicabilidad mínima y firma obligatoria del profesional.
- Recoger los huecos del prototipo que la implementación tuvo que cubrir (no había pantalla de login ni catálogo de alimentos del nutricionista) como parte del análisis.
- La ampliación clínica de la ficha del cliente que propuso la auditoría externa (patologías, medicación, bioquímica; más de 25 variables) se cita aquí como requisito identificado y se remite a trabajo futuro: el modelo v0 no la incorpora de forma consciente.

Fuente: capítulo actual + auditoría externa de diseño.

Capítulo 4. Diseño del sistema

Capítulo nuevo: la arquitectura, el modelo de datos y las decisiones que conectan todas las piezas.

4.1 Arquitectura general - Tres capas: frontend Next.js (Server Components y Server Actions), PostgreSQL gestionado (Supabase) y backend Python (FastAPI); por qué la lógica de IA va en un servicio Python aparte y no en una función serverless (solver de larga duración, secretos, proceso persistente). - La decisión rectora: la tabla de restricciones como interfaz estable entre productores (formulario manual, LLM) y consumidor (solver). El vocabulario se cerró antes de tocar el LLM; la vía manual es respaldo permanente, no provisional.

Figura: diagrama de arquitectura en capas + diagrama de la interfaz productor/consumidor (tres productores, un consumidor).

4.2 Modelo de datos - Las 16 tablas agrupadas (catálogos, alimentos, personas, restricciones, planes, trazabilidad de IA, cola de tareas); evolución desde el modelo vo con las migraciones que añadió la implementación (citas, disponibilidad, mensajes, cola, vínculo del cliente con su cuenta). - Decisiones de modelado con su porqué: composición nutricional en EAV (alimento-nutriente-valor por 100 g) en vez de columnas planas; el plan sin tabla intermedia de comidas (jerarquía reconstruida en presentación); catálogo global y alimentos propios en la misma tabla; argumentos heterogéneos de las restricciones en JSONB sobre esquema fijo; estados derivados en lugar de columnas de estado (la fecha de firma decide borrador o firmado); claves primarias naturales compuestas donde la seguridad lo impone; métricas derivadas y no almacenadas (la adherencia se calcula, porque un porcentaje guardado envejece mal); la taxonomía funcional de familias de alimentos, con el dato ausente como opción honesta; soft-delete selectivo. - Límites de la capa de acceso y cómo condicionan el diseño: mutaciones multi-tabla sin transacción desde el frontend (compensación documentada) frente a la persistencia atómica del backend; el tope de filas del API REST que obliga a agregar los macros plan a plan.

Figura: diagrama entidad-relación agrupado por dominios. Fuente: el documento de diseño de la BD (bd-modelo-vo) + migraciones reales.

4.3 Seguridad y multi-tenancy - Autenticación con cookie de sesión, guardas de servidor por rol y la RLS como última línea de defensa: forzar URLs ajenas devuelve cero filas por la base de datos, no por la interfaz. - Dos roles sobre las mismas tablas resuelto solo con policies (las permisivas se combinan con OR: el segundo rol se añadió sin reescribir el primero); cuándo basta una policy y cuándo hace falta una función con privilegios del sistema (cuando la fila mezcla columnas del actor con columnas ajenas); guardas contra suplantación; qué

dato puede autorizar y cuál no (los metadatos editables por el usuario, nunca). - La firma como frontera de visibilidad: el cliente solo existe para planes firmados, en la capa de datos. - Reglas de negocio dentro de la policy (no marcar comidas de días futuros, no marcar comidas no planificadas). - La verificación: ocho tandas de evidencia empírica acumuladas durante el desarrollo, hasta 106 comprobaciones multi-rol de lectura y escritura ejecutables en un comando. - Separación de claves: las públicas viajan al navegador y solo pueden lo que la RLS permite; la clave de servicio no sale del backend. Y aunque el backend opera con privilegios, re-verifica la propiedad del cliente antes de generar: segunda línea de defensa coherente con el modelo.

Figura: diagrama de la RLS multi-rol (dos roles, mismas tablas, policies y funciones).

4.4 Despliegue - Cuatro patas: frontend en Netlify, base de datos en Supabase (Frankfurt), backend en Render (Frankfurt) y el worker de generación en la máquina del profesional, porque ahí vive el LLM local: la decisión de privacidad condiciona la topología. - Límites del hosting gratuito, asumidos y medidos: arranque en frío del backend, límite de tiempo del solver calibrado contra la CPU de producción (no la de desarrollo) y paralelismo del solver configurable por entorno tras un incidente de memoria. - Mitigaciones legales del prototipo público: aviso de uso académico, datos de demostración seudonimizados, credenciales fuera del repositorio; repositorio privado sin protección de rama como limitación asumida.

Figura: diagrama de despliegue con las cuatro patas y el flujo de datos.

4.5 Metodología de desarrollo - Desarrollo iterativo por bloques (BD, conexión, frontend, motor, IA, panel del cliente), cada uno cerrado con verificación propia; los bancos de pruebas quedan como permanentes y se re-ejecutan al cerrar cada bloque. - El prototipo Figma como contrato inicial y la auditoría diseño-implementación como contribución metodológica: 169 diferencias inventariadas (40 altas, 65 medias, 64 bajas), resueltas como actualizar diseño (61), actualizar código (28) o justificar en memoria (80). - La filosofía “versión mínima que aplique las features” demostrada con datos: la aplicación solo pinta lo que tiene tabla detrás (ejemplos por pantalla), con recortes honestos en lugar de funcionalidad fingida. - Honestidad sobre la deuda detectada: frames del prototipo que arrastraban el alcance anterior al pivote, y el drift cromático que argumenta a favor de tokenizar también el diseño.

Fuente: la auditoría diseño-implementación (documento interno con los 169 items) + las decisiones registradas al cerrar cada bloque. Aquí se integra la justificación de las divergencias conscientes que quedó pendiente de redactar.

4.6 Patrones del frontend - Server Components que leen directamente bajo la sesión del usuario, sin capa API intermedia: la RLS sustituye al filtrado manual. - Un único login para

los dos roles con reparto en la raíz y guardas espejo por sección; el panel del cliente vive bajo su propio prefijo de rutas. - Server Actions de mutación con validación en servidor y errores devueltos a la interfaz; validación flexible frente a estricta (el aviso que no bloquea: el profesional es soberano de su agenda). - Un único formulario adaptativo para 13 tipos de restricción, con la metadata y la validación extraídas a módulos puros; el mismo formulario sirve a tres contextos distintos cambiando propiedades. - Actualización por sondeo (mensajería a 15 s, generación a 4 s) como decisión deliberada frente a websockets; agregación de macros sobre el modelo EAV con manejo explícito de datos incompletos.

Fuente: código del frontend + decisiones registradas por bloque.

Capítulo 5. El motor de generación

Capítulo nuevo. Es el corazón técnico y el de fuente más cocinada: el documento de formalización (solver-formalizacion-v1) se trasvasa casi directo, con la notación ya pensada para LaTeX. Sigue la forma del capítulo 9 del TFG de Adrián que me compartisteis (conjuntos, parámetros, variables con dominio, restricciones numeradas, objetivo, estrategia), tomando la forma y no la tecnología.

5.1 Formulación del problema - Conjuntos (alimentos, días, comidas, nutrientes), parámetros precalculados en Python (necesidades por Mifflin-St Jeor, escala entera) y variables de decisión: presencia binaria más gramos enteros, con el enlace que evita raciones testimoniales.

5.2 Restricciones estructurales (R1 a R8) - Las invariantes que se aplican siempre: presencia mínima, tope de items, suelo calórico, rangos humanos de macros, no repetir alimento el mismo día, variedad diaria y semanal, reparto entre días. - Dos decisiones con sustancia: el suelo calórico es el metabolismo basal y no el gasto total (permite déficit sin permitir planes peligrosos), y el reparto es blando a propósito (mejor un plan factible imperfecto que un infactible). - De la validez matemática a la lógica nutricional: el problema observado (planes que concentraban un alimento) y la evidencia antes/después de introducir las reglas.

Figura: tabla o gráfica antes/después de las reglas estructurales.

5.3 El catálogo de restricciones configurables - Los 16 tipos con su formulación, y la distinción dura/blanda como eje del modelo (naturaleza del tipo, prioridad de la fila, peso que modula la penalización).

5.4 Función objetivo y resolución - Penalizaciones ponderadas con linealización del valor absoluto (el mismo truco del TFG de Adrián), límite de tiempo, gap del 2 % y la política

de devolver factible aunque no haya óptimo demostrado.

5.5 Diagnóstico de infactibilidad - Del fallo opaco al subconjunto mínimo de restricciones en conflicto (assumptions del solver) con sugerencia legible: la base sobre la que luego se construye la explicación en lenguaje llano.

5.6 El validador determinista - Segundo par de ojos independiente: recalcula la nutrición desde la base de datos sin fiarse del solver y re-verifica las garantías; dos motores deterministas que deben coincidir. - Rechaza solo lo garantizado y avisa de lo no modelado (topes de micronutrientes): la distinción entre plan inseguro y plan mejorable, con el porqué (forzar el rechazo de lo no garantizado haría el veredicto no determinista).

5.7 API y persistencia - FastAPI como capa fina sobre funciones puras; contrato JSON de petición y respuesta (factible e infactible); persistencia del plan y sus items en una transacción atómica (contraste con las mutaciones del frontend); la firma como acto del profesional: el sistema persiste borradores, solo el colegiado los convierte en definitivos. - La verificación de este capítulo: banco de 12 casos (6 reales, 6 sintéticos) corrido contra la API desplegada antes de añadir el LLM.

Figura: ninguna nueva (las ecuaciones son el contenido). Fuente: solver-formalizacion-v1 (formulación completa), especificación vo (casos de prueba, métricas, contrato JSON), código del solver y validador.

Capítulo 6. El copiloto de IA

Capítulo nuevo. Abre con el marco y recorre el pipeline en el orden en que lo atraviesa un mensaje real.

6.1 Marco: editor inteligente, no generador autónomo - La formulación de la auditoría externa con perspectiva clínica: asistente de redacción para el profesional colegiado; las métricas que importan son tiempo ahorrado y control mantenido, no la calidad de una dieta autónoma. Posicionamiento legal explícito.

Figura: diagrama del pipeline completo (mensaje, validación previa, traducción, arbitraje del profesional, solver, validador, firma). Es la figura central de la memoria.

6.2 El traductor en dos capas - El LLM solo extrae intención sobre un menú cerrado de tipos y códigos; una capa determinista resuelve nombres contra el catálogo real y valida. Todo lo verificable en código se saca del LLM: menos superficie de alucinación, y lo no resoluble se rechaza en vez de adivinarse. - Hallazgos empíricos del modelo local pequeño (campos

opcionales omitidos del esquema, confusiones concretas) como evidencia de primera mano del diseño defensivo.

6.3 La elección del motor LLM - LLM local preferente: el dato clínico no sale a un tercero (RGPD) y el coste es cero; la interfaz de cliente LLM deja el motor remoto como alternativa comparable. La tensión con las APIs gratuitas que entrenan con los prompts, y el camino a producción (retención cero o servidor propio). - Cada traducción deja traza en la base de datos (entrada, salida, modelo, tokens, latencia, coste): la materia prima de la evaluación.

6.4 Validación previa del mensaje - Puerta determinista de idioma y tres comprobaciones LLM aisladas (ámbito, completitud, contradicciones), cada una con contexto limpio y con el modelo mínimo que hace bien ese trabajo, elegido por benchmark propio. - Tres capas de defensa contra la contradicción, cada problema en la capa más barata que puede resolverlo: el LLM lo categorial, el código puro los conflictos entre peticiones y contra lo ya guardado, el solver lo aritmético. - La infactibilidad en lenguaje llano: del subconjunto en conflicto a una frase con nombres reales del catálogo.

6.5 Generación asíncrona - La cola de tareas vive en la propia base de datos: el frontend encola y responde al instante; el worker reclama la tarea de forma atómica (sin doble procesado aunque haya varios workers) y la mueve por los estados encolada, en proceso, hecha o fallida. Sin broker externo: la versión mínima frente a Celery/Redis, y material de BD (bloqueos, índice parcial). - El worker corre en la máquina del profesional porque ahí vive el LLM local; la API solo encola y sirve el estado. Consecuencia honesta: sin el worker arrancado no se procesa, y la interfaz lo comunica. - Contraste con la primera versión síncrona (bloqueaba la pantalla de 15 a 90 s); el sondeo reutiliza el patrón de mensajería. Robustez: reconexión con reintentos, ninguna tarea se procesa dos veces. - El worker no reimplementa nada: encadena traductor, solver, validador y persistencia por el mismo camino que la vía síncrona.

Figura: diagrama del flujo asíncrono (frontend, cola, worker, sondeo) con la máquina de estados.

6.6 El arbitraje del profesional - El punto donde la IA propone y el colegiado dispone, restricción a restricción: guardar en el perfil del cliente, aplicar solo a esta generación o descartar. Nada se persiste sin decisión humana, y lo que el LLM no entendió se muestra, no se oculta. - La lista unificada donde convergen las dos vías (IA y manual) con su origen real registrado; la entrada manual convive en la misma pantalla, incluida la generación sin IA. - Ningún fallo silencioso: cada veredicto de la validación previa tiene representación visual con su mensaje y su camino de corrección; las generaciones fallidas quedan visibles con su explicación (antes desaparecían sin rastro: el feedback de error es una feature que se diseña). - La capa de lenguaje llano: una única fuente de frases naturales para las restricciones,

compartida por la ficha del cliente, los ajustes y el arbitraje; el selector manual se organiza por intención del profesional, no por nombre técnico del tipo.

6.7 El estilo clínico del profesional - Las reglas que el nutricionista declara una vez y se aplican a todos sus planes: mismo vocabulario, tercer productor, y la verificación de que el solver las honra en un plan de un cliente que no las tiene. - La versión mínima frente a la visión ambiciosa de la auditoría (doble representación con vector, cuestionario de arranque, aprendizaje por correcciones), que queda como línea de trabajo futuro.

Fuente: código de traductor, validación previa, worker y pantallas + decisiones registradas por bloque. No hay documento único cocinado: es el capítulo con más redacción nueva.

Capítulo 7. Evaluación

Capítulo nuevo. Reúne lo medido durante el desarrollo y la prueba con usuarios.

7.1 Estudio de formatos de mensaje - Pregunta: si el formato del texto del nutricionista afecta a la traducción y si merece la pena un módulo reescritor. Diseño pre-registrado (criterio de decisión fijado antes de medir): 5 formatos, 13 mensajes con verdad de referencia, 3 repeticiones, 195 llamadas a temperatura 0. - Hallazgo negativo: la mejora observada no alcanza el umbral pre-registrado, así que el reescritor no se construye. El hallazgo se defiende como resultado: el traductor es robusto al formato por arquitectura (catálogo cerrado, salida estructurada, capa determinista), y el criterio previo impide ajustar la conclusión a posteriori.

Figura: las 6 gráficas del estudio, ya generadas en PDF para LaTeX (acierto por mensaje, exhaustividad y precisión, latencia, tokens, mapa mensaje-formato, paso de las validaciones previas).

7.2 Comportamiento de modelos locales pequeños - El hilo de hallazgos de primera mano acumulado: eco de los ejemplos del prompt, el razonamiento previo al veredicto forzado por esquema, sensibilidad al framing e incluso a las tildes, y la matriz modelo por prompt que atribuyó un falso positivo al modelo y no al prompt. - La conclusión matizada: “el modelo mínimo que hace bien cada trabajo” exige bancos que cubran las clases de mensaje reales, porque el mínimo puede ser más grande de lo medido.

7.3 Rendimiento y escalado medidos - El coste de crecer el catálogo (28 a 58 a 100 alimentos): de óptimo demostrado a factible con incumbente, con tiempos; en el hosting gratuito, tres efectos medidos, incluida la distinción entre responder y diagnosticar (demostrar una infactibilidad se encarece más que resolverla). - El estilo clínico tiene coste: dos términos blandos más bastaron para agotar el límite de tiempo en la CPU gratuita, medido con sonda; el paralelismo del solver adaptado a los recursos de la instancia sin

degradar la calidad del plan. - Metodología: línea base antes de tocar nada para poder atribuir degradaciones, y repeticiones en lugar de pasadas sueltas. - Resumen de verificación continua del proyecto: los bancos permanentes (solver, validación previa, aislamiento multi-rol, extremo a extremo) y los datos de demostración generados por el propio pipeline, no escritos a mano.

7.4 Prueba con nutricionistas (pendiente de realizar) - Protocolo: prueba remota con tareas guiadas sobre los requisitos, ambos roles, cuenta compartida y ventanas anunciadas de disponibilidad de la IA. - Conclusiones y pautas de mejora redactadas (no necesariamente implementadas) como última retroalimentación del ciclo usuario real, prototipo, producto, usuario real.

Fuente: el documento del estudio de formatos (completo, con datos crudos reproducibles) + mediciones registradas al cerrar cada bloque + la prueba de usuario cuando se realice.

Capítulo 8. Conclusiones y trabajo futuro

Hoy es una cáscara. Contenido previsto:

- Balance contra los objetivos: el ciclo completo funciona (dos roles, plan firmado, seguimiento del cliente de vuelta al profesional) y el copiloto está integrado de punta a punta con el motor determinista como red de seguridad.
- Las lecciones transversales: cerrar el vocabulario antes que el LLM, verificar cada capa por separado, los límites del hosting gratuito como condicionante de diseño, y el valor del hallazgo negativo.
- Trabajo futuro, ya justificado durante el desarrollo: edición de planes y alimentos, notas privadas del cliente (requisito conservado y aplazado), calendario en rejilla, mensajería en tiempo real, validación clínica por patología y fármaco (con su dependencia explícita del modelo de datos), carga masiva de BEDCA/USDA, la visión ambiciosa del estilo clínico, motor LLM remoto con retención cero, comparativa multi-solver, workers en la nube y hosting de pago.
- Limitaciones honestas: transaccionalidad limitada desde el frontend, fechas en UTC, caducidad de los datos de demostración, dependencia del worker local.

Fuente: todo lo anterior; se escribe al final.

Anexos

- **A. Catálogo original de requisitos (V1):** ya escrito. Los 61 RF y 7 RNF originales como registro del pivote.
- **B. Requisitos V2 detallados:** los 29 RF con descripción, especificación y justificación (hoy el fichero es placeholder).
- **C. Formalización completa del solver:** la tabla de los 16 tipos con su modelado y el mapeo restricción a función del código (material de defensa verificable).
- **D. Prompts del sistema:** los prompts del traductor y de las tres validaciones previas, con sus esquemas de salida y ejemplos.
- **E. Evaluación experimental:** datos crudos del estudio de formatos, guía de la prueba con nutricionistas y sus resultados.
- **F. Auditoría externa de diseño:** los tres informes independientes (BD, IA, clínica), la síntesis y las 15 decisiones que cerraron el diseño.

Qué falta por tener antes de completar la memoria

- **La prueba con nutricionistas:** alimenta 7.4 y parte de las conclusiones. Todo lo demás del capítulo 7 puede escribirse ya.
- **El cierre de la integración y la demo:** puede aportar ajustes menores al capítulo 4 (despliegue) y a las conclusiones.
- **La bibliografía completa:** las referencias del solver (8), las de la investigación de BD y las de competidores están identificadas; falta consolidarlas en el fichero de bibliografía.